

Version 0.2.0 | Normative reference

A normative method for engineering accountability, traceability,
and diagnosable failure

Rafael Rodríguez Ramírez

Generated 2026-01-18 12:05 UTC

Explicit Software Engineering Method



Contents

Index	4
Prologue - Context and Motivation	6
0. The Problem	7
0.1 Situation	7
0.2 Problem Statement	7
0.3 Hypothesis	7
0.4 Objective	7
0.5 Scope and Non-Objectives	7
1. Why This Is a Method	8
1.1 Methodology vs Method vs Process	8
1.2 Why “Method” Fits	8
1.3 Dependency Without Rigidity	8
1.4 Separation of Concerns	8
1.5 Method as Evaluative Function	8
2. From Software to Software Product	9
2.1 Software Is Output; a Product Is an Outcome	9
2.2 Engineering Makes the Difference	9
2.3 Products Are Diagnosable; Code Alone Is Not	9
2.4 Process Does Not Create Products by Itself	9
2.5 The Role of Explicitness	10
2.6 Summary	10
3. Normative Core Rules (Draft)	11
3.0 How to Read These Rules	11
3.1 Rule 1 - Intent Declaration (Mandatory)	11
3.2 Rule 2 - Decision Explicitness (Mandatory)	11
3.3 Rule 3 - Artifact Traceability (Mandatory)	11
3.4 Rule 4 - Validation Requirement (Contextual)	12
3.5 Rule 5 - Assumption Declaration (Mandatory)	12
3.6 Rule 6 - Explicit Compensation (Mandatory)	12
3.7 Rule 7 - Parameter Awareness (Mandatory)	12
3.8 Rule 8 - Diagnosability of Failure (Outcome Rule)	12
3.9 Meta-Note	13
3.10 Conclusion	13
4. Relationship to Standards	14
4.1 Compatibility with IEEE and ISO Standards	14
4.2 Process Frameworks vs Method Execution	14
5. Orthogonality to Process Frameworks	15
5.1 Different Problem Spaces	15
5.2 Logical Dependencies vs Process Prescription	15
5.3 Coexistence with Process Structure	15
5.4 What Orthogonality Does and Does Not Mean	15
5.5 Summary	15
6. Failure as a Method Outcome	16
6.1 Freedom of Organization, Not Freedom of Opacity	16
6.2 Failure Is Not a Surprise	16
6.3 Failure as Evidence, Not Narrative	16
6.4 Method-Level Responsibility	16
6.5 Why Failure Improves Engineering	17
6.6 Summary	17

7. Formalization Sketch - Explicit Software Engineering Method	18
7.1 Universe of Discourse	18
7.1.1 Entity Sets	18
7.1.2 Predicates (Typing)	18
7.2 Core Relations (Graph Model)	18
7.2.1 Fundamental Relations	18
7.2.2 Traceability (Derived)	19
7.3 First-Order Logic (FOL) Axioms / Invariants	19
7.3.1 Rule 1 - Intent Declaration (Mandatory)	19
7.3.2 Rule 2 - Decision Explicitness (Mandatory)	19
7.3.3 Rule 3 - Artifact Traceability (Mandatory)	19
7.3.4 Rule 4 - Validation Requirement (Contextual)	20
7.3.5 Rule 5 - Assumption Declaration (Mandatory)	20
7.3.6 Rule 6 - Explicit Compensation (Mandatory)	20
7.3.7 Rule 7 - Parameter Awareness (Mandatory)	20
7.3.8 Rule 8 - Diagnosability of Failure (Outcome Rule)	20
7.4 Deontic Layer (Obligations, Permissions, Consequences)	20
7.4.1 MUST as Obligation	20
7.4.2 Compensation as Permission Under Obligation	21
7.4.3 Consequences (Violation Semantics)	21
7.5 Minimal Compliance and Diagnosability as Functions	21
7.5.1 Compliance Function	21
7.5.2 Diagnosability Function	21
7.6 Practical Notes (Non-Normative)	21
Glossary	22

Index

- Prologue - Context and Motivation
- 0. The Problem
 - 0.1 Situation
 - 0.2 Problem Statement
 - 0.3 Hypothesis
 - 0.4 Objective
 - 0.5 Scope and Non-Objectives
- 1. Why This Is a Method
 - 1.1 Methodology vs Method vs Process
 - 1.2 Why “Method” Fits
 - 1.3 Dependency Without Rigidity
 - 1.4 Separation of Concerns
 - 1.5 Method as Evaluative Function
- 2. From Software to Software Product
 - 2.1 Software Is Output; a Product Is an Outcome
 - 2.2 Engineering Makes the Difference
 - 2.3 Products Are Diagnosable; Code Alone Is Not
 - 2.4 Process Does Not Create Products by Itself
 - 2.5 The Role of Explicitness
 - 2.6 Summary
- 3. Normative Core Rules (Draft)
 - 3.0 How to Read These Rules
 - 3.1 Rule 1 - Intent Declaration (Mandatory)
 - 3.2 Rule 2 - Decision Explicitness (Mandatory)
 - 3.3 Rule 3 - Artifact Traceability (Mandatory)
 - 3.4 Rule 4 - Validation Requirement (Contextual)
 - 3.5 Rule 5 - Assumption Declaration (Mandatory)
 - 3.6 Rule 6 - Explicit Compensation (Mandatory)
 - 3.7 Rule 7 - Parameter Awareness (Mandatory)
 - 3.8 Rule 8 - Diagnosability of Failure (Outcome Rule)
 - 3.9 Meta-Note
 - 3.10 Conclusion
- 4. Relationship to Standards
 - 4.1 Compatibility with IEEE and ISO Standards
 - 4.2 Process Frameworks vs Method Execution
- 5. Orthogonality to Process Frameworks
 - 5.1 Different Problem Spaces
 - 5.2 Logical Dependencies vs Process Prescription
 - 5.3 Coexistence with Process Structure
 - 5.4 What Orthogonality Does and Does Not Mean
 - 5.5 Summary
- 6. Failure as a Method Outcome
 - 6.1 Freedom of Organization, Not Freedom of Opacity
 - 6.2 Failure Is Not a Surprise
 - 6.3 Failure as Evidence, Not Narrative
 - 6.4 Method-Level Responsibility
 - 6.5 Why Failure Improves Engineering
 - 6.6 Summary
- 7. Formalization Sketch - Explicit Software Engineering Method

- 7.1 Universe of Discourse
 - * 7.1.1 Entity Sets
 - * 7.1.2 Predicates (Typing)
 - 7.2 Core Relations (Graph Model)
 - * 7.2.1 Fundamental Relations
 - * 7.2.2 Traceability (Derived)
 - 7.3 First-Order Logic (FOL) Axioms / Invariants
 - * 7.3.1 Rule 1 - Intent Declaration (Mandatory)
 - * 7.3.2 Rule 2 - Decision Explicitness (Mandatory)
 - * 7.3.3 Rule 3 - Artifact Traceability (Mandatory)
 - * 7.3.4 Rule 4 - Validation Requirement (Contextual)
 - * 7.3.5 Rule 5 - Assumption Declaration (Mandatory)
 - * 7.3.6 Rule 6 - Explicit Compensation (Mandatory)
 - * 7.3.7 Rule 7 - Parameter Awareness (Mandatory)
 - * 7.3.8 Rule 8 - Diagnosability of Failure (Outcome Rule)
 - 7.4 Deontic Layer (Obligations, Permissions, Consequences)
 - * 7.4.1 MUST as Obligation
 - * 7.4.2 Compensation as Permission Under Obligation
 - * 7.4.3 Consequences (Violation Semantics)
 - 7.5 Minimal Compliance and Diagnosability as Functions
 - * 7.5.1 Compliance Function
 - * 7.5.2 Diagnosability Function
 - 7.6 Practical Notes (Non-Normative)
- Glossary

Prologue - Context and Motivation

Most engineers have seen systems that work and still feel fragile. Things compile, deploy, and pass tests, yet the codebase seems to depend on a handful of unwritten decisions and fragile expectations. Changes land with confidence, until they do not, and the reason is hard to state without reconstructing the past from memory.

Many of the most consequential decisions are made under pressure. They solve real constraints, but later those choices become invisible. The system carries them forward, while the people who made them move on, and the reasoning fades. When failures happen, the explanations often sound like stories: who was in the room, which deadline mattered, what seemed reasonable at the time.

Software development looks mature from the outside. We have reliable tooling, experienced teams, and established processes. Yet the engineering reasoning that produced the system often remains implicit. The process is visible; the accountability of the reasoning is not. Something important was never written down, and we feel it most when the system is stressed.

This is not a problem of effort or intelligence. It is a problem of visibility. The work is real, but the reasoning behind it disappears. Without a way to preserve that reasoning, we lose our ability to understand failures as engineering outcomes rather than as narrative justifications.

The method described here is a response to that gap. It aims to make engineering work explicit, to preserve reasoning over time, and to make failures understandable in technical terms. It is not a process or a framework. It is a method, offered to those who recognize the problem and want a more accountable way forward.

0. The Problem

0.1 Situation

Software development practice has matured in tooling, coordination, and delivery discipline, yet a persistent gap remains. Systems are often delivered that function, but they do not qualify as accountable engineering products because the reasoning that shaped them is not preserved.

Failures are common and difficult to diagnose objectively. Post-failure explanations tend to rely on narrative rather than evidence, even in organizations that adopt recognized standards, agile frameworks, and best practices.

0.2 Problem Statement

The situation above leads directly to a single core problem: the absence of explicit, enforceable representation of engineering reasoning. Intent, decisions, assumptions, trade-offs, and validation rationale remain implicit or informally captured, which prevents objective evaluation, diagnosis, and learning.

This is not a problem of missing processes or tools. Processes coordinate work, but they do not make engineering reasoning accountable on their own.

The consequences of implicit reasoning include: - decisions that cannot be evaluated post hoc - failures that cannot be diagnosed in engineering terms - learning that remains episodic rather than cumulative

0.3 Hypothesis

Given the core problem, the hypothesis follows: if engineering reasoning is made explicit and enforceable, independent of process or workflow, then software outcomes can be evaluated as engineered products and failures can be diagnosed with accountability rather than narrative.

0.4 Objective

The objective is to define a minimal, normative, process-orthogonal method that specifies obligations and consequences and makes engineering work objectively evaluable.

0.5 Scope and Non-Objectives

This work does not attempt to replace organizational planning, coordination, or delivery practices. It distinguishes engineering accountability from organizational planning and addresses only the former.

Uncertainty and subjectivity are not eliminated; they are made explicit and preserved as part of the engineering record.

This problem cannot be solved by adding more processes or tools. It requires a different kind of intervention, which motivates the choice of a method as the next step.

1. Why This Is a Method

This work is intentionally framed as a method because it targets engineering accountability, not workflow coordination or academic justification. The distinction is operational: it determines what can be evaluated and how compliance can be assessed across different organizations and practices. A process can organize work, but a method determines whether the engineering result is accountable.

1.1 Methodology vs Method vs Process

Process is an ordered sequence of activities that supports coordination and delivery. Methodology is the study and justification of methods, providing conceptual explanations and rationale. A framework offers a reusable organizational structure that packages roles, practices, and conventions, while a philosophy provides guiding principles that shape judgment without imposing obligations. Each of these can improve organization and understanding, but none of them, by themselves, enforce engineering accountability.

For example, a team can start from code or from models and still follow the same process; the process does not determine whether the reasoning behind those choices remains accountable.

1.2 Why “Method” Fits

A method provides normative obligations and consequences for missing or insufficient engineering work. It makes evaluability independent of execution order, so work can be assessed even when teams follow different workflows or operate under different organizational constraints. This makes the method applicable across heterogeneous contexts without prescribing how work must be scheduled.

That distinction matters when requirements are discovered late. A process can accommodate late discovery, but a method determines whether the resulting changes are still accountable.

1.3 Dependency Without Rigidity

Engineering has logical dependencies: a problem must be understood before a solution can be validated. Acknowledging those dependencies does not require a fixed process sequence. The method constrains outcomes and relationships, not activity order, and allows teams to discover and revise understanding without violating accountability.

This is visible when decisions are made under pressure. The method does not forbid them; it requires that their consequences remain traceable and accountable even when timing is constrained.

1.4 Separation of Concerns

This approach is not a rejection of Agile, Scrum, RUP, or any other organizational approach. It separates concerns: processes organize work; a method defines the obligations that make engineering outcomes accountable. The method can coexist with any organizational approach because it does not prescribe how teams should coordinate.

The separation is practical: organizational choices optimize coordination, while engineering accountability requires a stable basis for evaluation regardless of coordination style.

1.5 Method as Evaluative Function

At its core, a method can act as an evaluative function: given engineering artifacts and context, it can determine whether obligations were met and where accountability is missing. This perspective motivates a method over process and prepares the ground for distinguishing software that merely works from software that qualifies as an engineered product.

Taken together, these points show why a method is the correct abstraction: it preserves accountability across different ways of organizing work and makes engineering outcomes evaluable. With that foundation, the next step is to clarify the boundary between software that functions and software that qualifies as an engineered product.

2. From Software to Software Product

Software and software products are not equivalent. Software consists of executable code, configurations, and data that produce behavior. A software product is the result of engineering: software whose behavior, constraints, and purpose can be explained, validated, and justified. This chapter focuses on the difference between working functionality and accountable engineering outcomes.

2.1 Software Is Output; a Product Is an Outcome

Software can exist without engineering intent. It may function, pass tests, or satisfy immediate needs, yet remain opaque with respect to why it exists in its current form, which decisions shaped it, which assumptions it relies on, which trade-offs were accepted, and how its correctness or fitness was established. A software product, by contrast, is an outcome whose properties are intentional and explainable.

Consider a data pipeline that has grown through urgent fixes. It produces correct outputs most days, but no one can explain why certain thresholds exist or which failure modes were accepted. The system works, yet its behavior cannot be justified without reconstructing past decisions from memory.

By contrast, a customer-facing service may behave within defined constraints, and when a latency regression appears, the team can trace the change to a specific decision and the condition it was meant to satisfy. The service still fails in real conditions, but the failure is diagnosable as an engineering outcome.

The distinction is not about size, polish, performance targets, or bug counts. It is about engineering accountability.

2.2 Engineering Makes the Difference

Engineering introduces structure between problem and solution. A product is not defined by having more features or fewer bugs; it is defined by the visibility and accountability of the reasoning that shaped it. This is why a prototype can be valid software while still not qualifying as a product: it may demonstrate behavior without preserving the engineering reasoning behind it.

Once accountability exists, diagnosability becomes a property of the product rather than a consequence of who remembers the history.

2.3 Products Are Diagnosable; Code Alone Is Not

A defining property of an engineered product is diagnosability. Success in production does not retroactively create engineering accountability; it only shows that the system is currently acceptable. When failures occur, an engineered product allows the team to explain the failure in terms of the reasoning that led to the behavior, rather than relying on retrospective narrative or individual recollection.

This distinction explains why organization alone cannot close the gap.

2.4 Process Does Not Create Products by Itself

Processes coordinate activities. They do not guarantee that engineering reasoning is preserved. A team can follow a well-defined process and still produce software whose behavior cannot be explained beyond surface defects. This is not a critique of process; it is a recognition that organization and accountability are distinct concerns.

The difference is structural, which is why it requires an explicit mechanism.

2.5 The Role of Explicitness

Explicitness is the mechanism by which software is elevated to a product. When reasoning is preserved, outcomes are intentional, quality becomes defensible in context, failures are explainable, and learning is cumulative rather than anecdotal. This is the structural difference between working software and an engineered product.

2.6 Summary

Software can exist without engineering; products cannot. A software product is defined by explicit reasoning, not by delivery mechanics. Processes may produce software; a method defines the obligations that allow software to qualify as a product.

This distinction motivates the need for explicit obligations. The next step is to define what those obligations are, without yet prescribing how teams organize their work.

3. Normative Core Rules (Draft)

These rules are normative constraints and obligations of the method. Each rule is stated to be verifiable, to imply consequences when violated, and to support analysis without dependence on tooling. They are independent of workflow sequencing and do not prescribe a process.

3.0 How to Read These Rules

These rules constrain outcomes, not activity order. A team may satisfy them through different workflows and still be compliant, because the method evaluates what exists and how it is justified, not when it was produced.

Missing artifacts are allowed only through explicit compensation. Compensation is an auditable act that records accepted risk; it does not remove the underlying obligation.

Violations are diagnosable conditions, not moral failures. The intent is to make gaps observable and explainable so they can be corrected or accepted explicitly.

3.1 Rule 1 - Intent Declaration (Mandatory)

Every non-trivial engineering change **MUST** declare an explicit Intent. Intent defines the desired outcome, property, or constraint the change aims to satisfy. Intent **MUST** have a defined scope. Changes without declared intent are considered implicit work.

This rule ensures that changes are anchored to a stated objective and can be evaluated against it, regardless of whether the work begins from code or from models.

Example: A team modifies caching behavior with the intent to reduce latency for a defined service boundary, and records that scope explicitly.

Rationale: Without intent, no validation or traceability is possible. Quality becomes accidental.

3.2 Rule 2 - Decision Explicitness (Mandatory)

Every irreversible or impactful engineering decision **MUST** be made explicit. A Decision **MUST** reference: - the Intent it serves - the alternatives considered (at least implicitly) Undocumented decisions are treated as assumptions by default.

This rule preserves rationale and ensures that impactful choices can be reviewed after the fact, even when those choices are made under time pressure.

Example: Under a production deadline, a team selects a simpler data model and records the decision along with the alternative that was rejected for risk reasons.

Rationale: Undocumented decisions prevent post-failure analysis and learning.



This diagram shows the minimal trace chain the rules expect: intent informs decisions, decisions shape artifacts, and validation ties artifacts back to their intended purpose.

3.3 Rule 3 - Artifact Traceability (Mandatory)

Every Artifact that affects system behavior **MUST** be traceable to at least one Intent or Decision. Orphan artifacts are considered unjustified artifacts. Traceability **MUST** be navigable in both directions.

This rule ensures that artifacts exist for a reason that can be located and reviewed, rather than surviving as unexplained residue of past work.

Example: A new configuration flag is linked to the decision that introduced it and the intent it was meant to satisfy.

Rationale: Untraceable artifacts increase complexity without accountable value.

3.4 Rule 4 - Validation Requirement (Contextual)

Artifacts and Decisions with critical impact **MUST** have explicit Validation. What is considered critical is a method parameter, not a fixed threshold. Validation **MAY** take different forms: - tests - reviews - proofs - simulations - external evidence

This rule ties validation effort to contextual criticality, allowing different forms of evidence while still requiring explicit justification for high-impact work.

Example: A high-risk change in a payment flow is validated through a targeted review and test evidence appropriate to its criticality.

Rationale: Critical work without validation is indistinguishable from speculation.

3.5 Rule 5 - Assumption Declaration (Mandatory)

Assumptions **MUST** be explicitly declared when evidence is missing or deferred. Assumptions **MUST** be identifiable as such. Assumptions **SHOULD** be tracked until validated or invalidated.

This rule makes uncertainty visible and prevents hidden premises from becoming silent dependencies in the system.

Example: A team assumes an external dependency will sustain a defined throughput until benchmarks can be run, and records that assumption explicitly.

Rationale: Implicit assumptions silently accumulate risk.

3.6 Rule 6 - Explicit Compensation (Mandatory)

When a required artifact or validation is missing, the absence **MUST** be explicitly compensated. Compensation **MUST**: - be documented - reference the missing element - state the accepted risk

Compensation is an explicit, auditable record of accepted risk. It does not erase the obligation; it marks a conscious deviation and allows evaluation of its impact.

Example: A hotfix is deployed without full validation, and the team records the missing validation, the accepted risk, and the follow-up plan.

Rationale: Shortcuts are allowed; silent shortcuts are not.

3.7 Rule 7 - Parameter Awareness (Mandatory)

The method **MUST** be applied with explicit contextual parameters. Examples of parameters: - domain criticality - risk tolerance - regulatory pressure - team scale - expected lifespan of the system

These parameters influence: - required explicitness level - validation depth - acceptable compensations

This rule ensures that obligations are applied with context, so evaluation reflects the actual risk and criticality of the work.

Example: A system in a regulated domain declares higher validation depth and stricter compensation limits than an internal prototype.

Rationale: Quality is contextual; rigor without context is waste.

3.8 Rule 8 - Diagnosability of Failure (Outcome Rule)

When failure occurs, it **MUST** be possible to explain it in terms of method application. At least one of the following **MUST** be identifiable: - missing intent - invalid decision - unvalidated artifact - false assumption - inadequate compensation - misconfigured parameters

This rule makes failure a traceable outcome of engineering choices, not a retrospective narrative. It enables analysis of which obligation was not met and why.

Example: After an outage, the analysis identifies an unvalidated artifact and a missing compensation record as the decisive gap.

Rationale: If failure cannot be explained in method terms, the method was not applied.

3.9 Meta-Note

These rules do not describe a process. They define constraints and obligations that must hold regardless of workflow. That is what makes them rules of a method, not of a process.

3.10 Conclusion

These rules make evaluation possible by specifying what must be explicit and how gaps are recorded. They transform failure into evidence by making missing intent, decisions, assumptions, validation, or compensation observable. Because they constrain outcomes rather than activity order, they remain independent of any specific process.

4. Relationship to Standards

The Explicit Software Engineering Method is designed to operate within software engineering standards rather than replace them. It assumes that standards define the context of work and clarifies how engineering accountability is established within that context.

4.1 Compatibility with IEEE and ISO Standards

IEEE / ISO software engineering standards define process frameworks, describing activities, roles, and lifecycle structures expected of engineering practice. They set the stage for what must be done, but they generally do not enforce explicit capture of engineering reasoning or ensure that rationale is preserved in an analyzable form.

This method complements those standards by constraining outcomes: it requires that engineering reasoning be explicit enough to support evaluation and diagnosis. Used together, standards define where engineering happens, while the method defines what must be true for those activities to count as accountable engineering.

It is possible to follow a standards-compliant process and still produce software that cannot be diagnosed after failure because the reasoning behind key choices was never made explicit. The same process can also produce an engineered product when intent and decisions are captured in a way that allows later evaluation of why the system behaves as it does.

4.2 Process Frameworks vs Method Execution

The method is orthogonal to process standards. It constrains engineering outcomes, not workflows, and allows the same process to be compliant or non-compliant depending on whether explicit reasoning is preserved. This separation means the method does not require changes to lifecycle phases or organizational routines.

This is not a conflict with Agile, plan-driven, or hybrid approaches. The method can be adopted incrementally within any of them, because it evaluates what exists rather than prescribing how work must be ordered or coordinated.

Compatibility with standards matters because it enables diagnosability and evaluation across different contexts. Standards can specify activities and artifacts, but they cannot by themselves explain failure as engineering evidence. The method fills that gap without altering the standard-defined structure of work.

The result is a clear division of responsibility: standards define where engineering happens, and the method defines what must be true for it to count as engineering.

5. Orthogonality to Process Frameworks

Orthogonality here is a structural property: the method evaluates engineering state independently of the sequence used to reach it. Order flexibility does not remove obligation; it only means there are multiple valid paths to a compliant outcome.

5.1 Different Problem Spaces

Process frameworks organize work through stages, roles, and coordination practices. They optimize how teams plan and deliver, but they do not determine whether the reasoning behind the work remains accountable. The method addresses that gap by evaluating what exists and how it is justified, regardless of how the work was scheduled.

5.2 Logical Dependencies vs Process Prescription

Engineering has logical dependencies, such as understanding a problem before validating a solution. Orthogonality does not deny those dependencies; it separates them from fixed sequencing. Real work is rarely linear: constraints are discovered late, assumptions are revised, and solutions evolve as understanding deepens.

One team might start from existing code, reverse-engineer the intent, and then record the reasoning that makes the current behavior defensible. Another team might start from models, define the intent first, and refine implementation until the behavior matches. Both can be compliant if the required reasoning is explicit and traceable. The method evaluates the state they reached, not the route they took.

5.3 Coexistence with Process Structure

A process can impose order while the method remains orthogonal. The method can be adopted within structured lifecycles because it does not require changes to phases or timelines. Compliance is evaluated on artifacts and traceability, not on whether a particular milestone happened first.

This makes incremental adoption possible: a team can introduce explicit reasoning in the parts of the system that matter most, without reorganizing the entire lifecycle.

5.4 What Orthogonality Does and Does Not Mean

Orthogonality does not mean anything goes. Obligations still apply, and missing reasoning remains a gap regardless of how the work was organized. The method is strict about outcomes while remaining flexible about paths.

5.5 Summary

Orthogonality makes the method applicable across teams, domains, and processes because it evaluates engineering state rather than activity sequence. It enables post-failure reconstruction by allowing accountability to be assessed even when the original work unfolded non-linearly. This is what allows the method to be universal without becoming prescriptive about workflow.

6. Failure as a Method Outcome

Complex systems fail. This is not an anomaly; it is a constant. The method does not aim to eliminate failure. It aims to make failure diagnosable as engineering evidence rather than as a narrative artifact.

The problem is not that failures occur. The problem is that they often cannot be explained objectively, even after the fact. What distinguishes engineering from trial-and-error is the ability to explain failure with traceable evidence. The method treats failure as an evaluable outcome of engineering reasoning, not an unexpected exception.

6.1 Freedom of Organization, Not Freedom of Opacity

Teams and organizations are free to organize their work as they see fit. The method does not interfere with coordination models, planning cadence, tooling, or internal structure. What it constrains is not how work is organized, but what must be explicit for the result to qualify as engineering. Freedom of organization is preserved; opacity is not.

6.2 Failure Is Not a Surprise

Most software failures are predictable in retrospect. After a failure, teams can usually reconstruct that the problem was not fully understood, assumptions turned out to be false, decisions were made under pressure, validation was insufficient, or trade-offs were accepted silently. The issue is not that these things happened; the issue is that they were not explicit.

This is where the distinction between “what happened” and “why this was possible” matters. The first can be reconstructed from logs and timelines. The second requires explicit reasoning artifacts that explain why a particular system state was allowed to exist.

6.3 Failure as Evidence, Not Narrative

Without explicit artifacts, failure is explained through narrative: memory, opinion, and hindsight rationalization. Postmortems often become stories about what happened rather than explanations of why it was possible. With the method applied, failure can be explained through evidence such as missing intent, undocumented decisions, invalid assumptions, absent validation, uncompensated gaps, or misconfigured parameters. This shift transforms failure from a story into a technical diagnosis grounded in traceability and explicit obligations.

In one case, a service outage is explained as “unexpected traffic.” The timeline is clear, but the explanation is weak. Under the method, the diagnosis points to a missing intent about capacity, an undocumented scaling decision, an invalid assumption about load shape, and the absence of validation or compensation for skipping capacity testing.

In another case, a late-stage defect is attributed to “rushed changes.” The narrative assigns pressure as the cause, but not the engineering basis. Under the method, the diagnosis shows that a critical change shipped without recorded validation, a compensation for that gap was never documented, and the risk tolerance parameter was misconfigured for the system’s expected lifespan.

6.4 Method-Level Responsibility

When a system fails, responsibility can be assessed at the level of the method: whether intent was declared and understood, decisions were explicit and justified, assumptions were identified and tracked, validation matched the risk, compensations were explicit, and contextual parameters were realistic. If these questions cannot be answered, the method was not fully applied. Failure, in that case, is not mysterious; it is expected.

This does not imply negligence. It indicates where the engineering record was incomplete and where obligations were not met or were accepted without explicit compensation. The method makes that gap visible so it can be evaluated and corrected.

6.5 Why Failure Improves Engineering

Because failure becomes explainable, it becomes reusable. Teams can learn across projects, improve decision-making, adjust explicitness levels, refine compensations, and evolve their use of the method. Failure stops being an endpoint and becomes input that tightens engineering accountability over time.

6.6 Summary

Failure is inevitable; opacity is optional. The method does not dictate how teams work, but it dictates what must be explicit for work to be engineering. When failure occurs, it must be explainable in method terms; engineering maturity grows from diagnosed failure, not avoided failure.

The method does not prevent all failures, and some failures remain irreducible or external. Its scope is the evaluation of engineering reasoning, not the elimination of adverse outcomes. In that sense, failure becomes a measurable signal of engineering state, and the next step is to formalize how that evaluation can be performed.

7. Formalization Sketch - Explicit Software Engineering Method

This document provides a mathematical and logical formalization sketch of the method. It is intentionally minimal and focuses on: (1) entities, (2) relations, (3) normative obligations, and (4) diagnosability. It describes constraints over explicit artifacts and relations, not a prescriptive workflow. The goal is evaluability and diagnosability, not mathematical completeness.

The formalization aims for **operational objectivity under explicit context**, not absolute objectivity.

Before stating obligations, we must fix a universe of discourse. Without a declared universe, constraints have no stable targets and evaluation becomes ambiguous.

7.1 Universe of Discourse

We model engineering work as a set of entities and typed relations.

7.1.1 Entity Sets

Let the following sets exist:

- **I** : set of *Intents*
- **D** : set of *Decisions*
- **A** : set of *Artifacts*
- **S** : set of *Assumptions*
- **T** : set of *Trade-offs*
- **V** : set of *Validations*
- **C** : set of *Compensations*
- **Ch** : set of *Changes* (engineering changes)
- **P** : set of *Parameters* (contextual inputs)

Define the global entity set:

- **E = I D A S T V C Ch P**

7.1.2 Predicates (Typing)

We use unary predicates to type entities:

- $\text{Intent}(x)$, $\text{Decision}(x)$, $\text{Artifact}(x)$, $\text{Assumption}(x)$, $\text{TradeOff}(x)$, $\text{Validation}(x)$, $\text{Compensation}(x)$, $\text{Change}(x)$, $\text{Parameter}(x)$

Engineering reasoning is captured by relationships among these entities, not by an imposed sequence. A graph model makes traceability explicit without prescribing order.

7.2 Core Relations (Graph Model)

Engineering reasoning is represented as a typed directed multigraph over E.

7.2.1 Fundamental Relations

- **declares** $\text{Ch} \times \text{I}$
 $\text{declares}(\text{ch}, \text{i})$ means change **ch** declares intent **i**.
- **serves** $\text{D} \times \text{I}$
 $\text{serves}(\text{d}, \text{i})$ means decision **d** serves intent **i**.
- **justifies** $\text{A} \times (\text{I} \text{ D})$
 $\text{justifies}(\text{a}, \text{x})$ means artifact **a** is justified by intent or decision **x**.
- **assumes** $(\text{D} \text{ A}) \times \text{S}$
 $\text{assumes}(\text{x}, \text{s})$ means entity **x** relies on assumption **s**.

- **tradesOff** $(D \ A) \times T$
 $\text{tradesOff}(x, t)$ means entity x involves trade-off t .
- **validates** $V \times (A \ D \ I)$
 $\text{validates}(v, x)$ means validation v provides evidence for entity x .
- **compensates** $C \times (I \ D \ A \ V)$
 $\text{compensates}(c, x)$ means compensation c explicitly compensates missing/insufficient x .
- **paramOf** $P \times Ch$
 $\text{paramOf}(p, ch)$ means parameter p applies to change ch (or project scope containing it).

7.2.2 Traceability (Derived)

Define a derived relation **traceableTo**(x, y) as the existence of a path from x to y using allowed relation edges. At minimum, the trace graph uses edges: - declares, serves, justifies, validates, assumes, tradesOff

Traceability requirements in the method constrain the existence of such paths and their navigability.

First-order logic is used here to express invariants over the graph. These are constraints to be evaluated, not computations to be executed.

7.3 First-Order Logic (FOL) Axioms / Invariants

The method is expressed as constraints that must hold regardless of workflow sequencing, reflecting orthogonality to process order. These constraints correspond directly to the normative rules defined earlier.

Below, $\forall$ means “for all” and $\exists$ means “there exists”.

Rules 1-3 establish minimal explicitness and traceability for intent, decisions, and artifacts.

7.3.1 Rule 1 - Intent Declaration (Mandatory)

For any non-trivial change, an intent must be declared:

- $\forall ch (\text{Change}(ch) \wedge \text{NonTrivial}(ch) \rightarrow \exists i (\text{Intent}(i) \wedge \text{declares}(ch, i) \wedge \text{Scoped}(i)))$

If a non-trivial change has no declared intent, it is implicit work:

- $\forall ch (\text{Change}(ch) \wedge \text{NonTrivial}(ch) \wedge \neg \exists i (\text{Intent}(i) \wedge \text{declares}(ch, i)) \rightarrow \text{ImplicitWork}(ch))$

7.3.2 Rule 2 - Decision Explicitness (Mandatory)

For any impactful decision, it must serve an intent (or be explicitly compensated):

- $\forall d (\text{Decision}(d) \wedge \text{Impactful}(d) \rightarrow (\exists i (\text{Intent}(i) \wedge \text{serves}(d, i)) \vee \exists c (\text{Compensation}(c) \wedge \text{compensates}(c, d))))$

And decisions that do not serve an intent are treated as assumptions unless compensated:

- $\forall d (\text{Decision}(d) \wedge \text{Impactful}(d) \wedge \neg \exists i (\text{Intent}(i) \wedge \text{serves}(d, i)) \rightarrow \exists s (\text{Assumption}(s) \wedge \text{AssumptionByDefault}(d, s)))$

7.3.3 Rule 3 - Artifact Traceability (Mandatory)

Any behavior-affecting artifact must trace to an intent or decision:

- $\forall a (\text{Artifact}(a) \wedge \text{AffectsBehavior}(a) \rightarrow (\exists x ((\text{Intent}(x) \vee \text{Decision}(x)) \wedge \text{justifies}(a, x)) \vee \exists c (\text{Compensation}(c) \wedge \text{compensates}(c, a))))$

Orphan artifacts are unjustified:

- $\forall a (\text{Artifact}(a) \wedge \text{AffectsBehavior}(a) \wedge \neg \exists x ((\text{Intent}(x) \vee \text{Decision}(x)) \wedge \text{justifies}(a, x)) \rightarrow \text{Orphan}(a))$

Rules 4-6 cover validation, assumptions, and compensation under contextual risk.

7.3.4 Rule 4 - Validation Requirement (Contextual)

Critical elements require validation, with criticality dependent on parameters:

- $x (\text{Critical}(x) \rightarrow (v (\text{Validation}(v) \text{ validates}(v, x)) \quad c (\text{Compensation}(c) \text{ compensates}(c, x))))$

Critical(x) is not fixed; it is derived from parameters:

- $x (\text{Critical}(x) \quad \text{CriticalUnderParams}(x, \text{ParamsScope}(x)))$

Parameters are inputs to evaluation. They shape what counts as critical and what depth of validation is required; they do not excuse obligations.

7.3.5 Rule 5 - Assumption Declaration (Mandatory)

Whenever evidence is missing/deferred, assumptions must be explicit:

- $x (\text{EvidenceMissing}(x) \rightarrow s (\text{Assumption}(s) \text{ assumes}(x, s) \quad \text{Explicit}(s)))$

7.3.6 Rule 6 - Explicit Compensation (Mandatory)

Whenever a required element is missing, compensation must exist and declare accepted risk:

- $x (\text{Required}(x) \quad \text{Missing}(x) \rightarrow c (\text{Compensation}(c) \text{ compensates}(c, x) \quad \text{StatesAcceptedRisk}(c)))$

Rule 7 makes context explicit so that evaluation is reproducible across teams and time.

7.3.7 Rule 7 - Parameter Awareness (Mandatory)

Each relevant scope must declare applicable parameters:

- $ch (\text{Change}(ch) \quad \text{NonTrivial}(ch) \rightarrow p (\text{Parameter}(p) \text{ paramOf}(p, ch)))$

Parameters affect obligations by defining thresholds and scopes; they are not tuning knobs for correctness.

Rule 8 ties the constraints to diagnosability, aligning the method with failure analysis.

7.3.8 Rule 8 - Diagnosability of Failure (Outcome Rule)

For any observed failure f, at least one method-level explanation must be identifiable:

- $f (\text{Failure}(f) \rightarrow r (\text{Reason}(r) \text{ ExplainsInMethodTerms}(r, f)))$

Minimal admissible reasons:

ExplainsInMethodTerms(r, f) implies at least one of: - MissingIntent(f) - InvalidDecision(f) - UnvalidatedArtifact(f) - FalseAssumption(f) - InadequateCompensation(f) - MisconfiguredParameters(f)

7.4 Deontic Layer (Obligations, Permissions, Consequences)

FOL invariants describe truth conditions. The method is normative: it includes obligations. The deontic operators O, P, and R are interpretive layers over the invariants; they do not replace or extend the underlying FOL constraints. We model this with a deontic operator:

- **O**() : it is obligatory that holds (MUST)
- **P**() : it is permitted that holds (MAY)
- **R**() : it is recommended that holds (SHOULD)

7.4.1 MUST as Obligation

Example (Rule 1):

- $O(\text{Change}(ch) \quad \text{NonTrivial}(ch) \rightarrow i (\text{Intent}(i) \text{ declares}(ch, i) \quad \text{Scoped}(i)))$

7.4.2 Compensation as Permission Under Obligation

The method permits shortcuts only if compensated:

- $P(\text{Missing}(x)) \text{ IF } O(c(\text{Compensation}(c) \text{ compensates}(c, x) \text{ StatesAcceptedRisk}(c)))$

This expresses: missing artifacts/validation may occur, but only under explicit compensation. Compensation is permission under obligation; it records accepted risk and does not remove the obligation.

7.4.3 Consequences (Violation Semantics)

Define a violation predicate:

- $\text{Violation}(\text{rule_k}, \text{scope})$

For example:

- $\neg i(\text{Intent}(i) \text{ declares}(\text{ch}, i)) \text{ implies } \text{Violation}(\text{Rule1}, \text{ch})$

A compliance check is then a function from a repository state to the set of violations:

- $\text{Violations}(\text{State}) = \{ (\text{rule_k}, \text{scope}) \mid \text{State} \text{ rule_k}(\text{scope}) \}$

This is the basis for tooling without prescribing workflow.

7.5 Minimal Compliance and Diagnosability as Functions

7.5.1 Compliance Function

Given a state containing entities and relations:

- **Compliant(State)** iff $\text{Violations}(\text{State}) = \emptyset$

7.5.2 Diagnosability Function

Given a failure f and a state:

- **Diagnosable(f, State)** iff $\exists r \text{ ExplainsInMethodTerms}(r, f)$

This can be implemented as querying the explicit graph for missing links, missing validation, missing compensation, or parameter mismatches. It is the formal counterpart to the diagnosability requirement discussed earlier.

7.6 Practical Notes (Non-Normative)

This level of formalization is sufficient to support objective evaluation, failure diagnosis, and tooling implementation without prescribing workflow. - This formalization is compatible with graph databases and constraint checking. - It is intentionally independent of any specific development process or lifecycle. - Temporal logics (e.g., TLA+) and execution semantics may be introduced later to model evolution, reconstruction post-hoc, or concurrency, but are intentionally excluded from the method core.

Glossary

- Artifact** A produced, versioned, and inspectable work product that affects system behavior and can be validated and traced to intent or decisions.
- Assumption** A statement treated as true for reasoning or planning when evidence is missing or deferred, and that must remain identifiable until validated or invalidated.
- Compensation** An explicit and auditable acceptance of risk due to a missing required artifact or validation, recorded without removing the underlying obligation.
- Contextual Parameters** Declared inputs that characterize the engineering context and shape the required level of explicitness and validation for a given scope.
- Decision** A committed engineering choice among alternatives, made explicit and linked to the intent it serves.
- Diagnosability** The ability to explain a failure in method terms using explicit evidence of intents, decisions, assumptions, validations, compensations, and parameters.
- Explicit** Engineering reasoning captured as traceable artifacts with stated intent and validation rather than remaining undocumented or implicit.
- Explicitness** The property of engineering work being made explicit in a form that is traceable, evaluable, and diagnosable.
- Failure** An observed deviation from intended or required behavior or constraints that can be analyzed against explicit engineering reasoning.
- Intent** The desired outcome, property, or constraint that a change or system must satisfy, with a defined scope.
- Method** A prescriptive and executable set of obligations and constraints over engineering artifacts and relationships, parameterizable and analyzable, and independent of workflow order.
- Method Application** The concrete application of the method to a specific scope with declared parameters and explicit artifacts such that compliance and diagnosability can be evaluated.
- Normative Rule** A mandatory statement of obligation that defines what must be explicit and the consequences of omission.
- Orthogonality** Independence from workflow order or lifecycle sequence while maintaining fixed engineering obligations.
- Process** An ordered sequence of activities or phases used to organize work, which may embed a method but does not define engineering obligations by itself.
- Software** Executable code, configurations, and data that produce system behavior.
- Software Product** Software whose behavior, constraints, and purpose are accountable through explicit intent, decisions, validation, and traceability.
- Traceability** Explicit and navigable links among intents, decisions, assumptions, artifacts, and validations that allow reasoning to be inspected and evaluated.
- Validation** Evidence that an artifact or decision satisfies its stated intent under the declared context.